

- 2) 1) El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo **Flow**.
- 2) El operador **withContext(Dispatchers.io)** permite ejecutar tareas pesadas fuera del main Thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos.
- 3) Para que un servicio en primer plano de ubicación sea válido en Android 10+ debe declararse en el manifiesto con el tipo **FOREGROUND_SERVICE_LOCATION**.
- 4) La función **AppContainer** de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".
- 5) El componente que permite transformar una API basada en callbacks antiguas en una función suspendible moderna se llama **Suspend Coroutine**.
- 6) El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente **AndroidManifest.xml**.
- 7) En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina **HAL (Hardware Abstraction Layer)**.
- 8) El operador de Flow que permite combinar múltiples puentes como GPS y sensores para emitir un estado unificado es **Combine**.

3) // 1. Repositorio y DI

```
class UsuarioRepository {  
    private val UsuarioDao : UsuarioDao  
} {  
    fun obtenerUsuarios () = usuarioDao.obtenerUsuarios ()  
}
```

```
class MyApp : Application () {
```

```
    val usuarioRepository : UsuarioRepository by lazy {  
        usuarioRepository (usuarioDao)  
    }
```

```
    private val UsuarioDao : UsuarioDao by lazy {  
        AppDatabase.getDatabase (this).usuarioDao ()  
    }
```

```
}
```

// 2. Lógica Dinámica de Identidad

```
- data class Usuario (  
    val nombre : String?,  
    val apellidos : String?,  
    val edad : Int?  
)
```

```
fun main () {  
    val usuario = Usuario (  
        nombre = "Miguel",  
        apellidos = "Sanchez Aznárez",  
        edad = 21  
    )
```

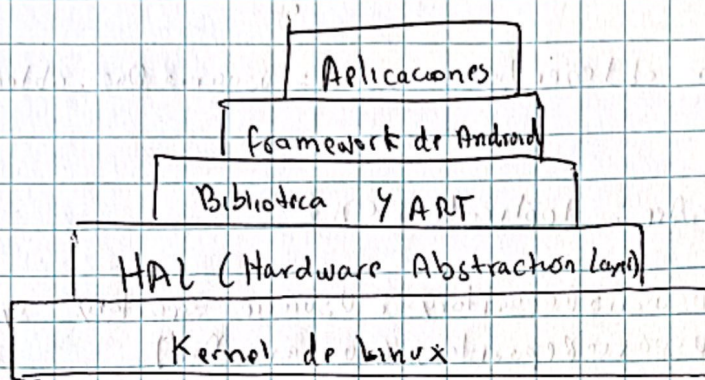
```
    val puntuajeIdentidad =  
        (usuario.nombre?.length?.toLong() +  
        usuario.apellidos?.length?.toLong())
```

```
    println ("Usuario: {usuario.nombre} {usuario.apellidos},  
    tu puntuaje de identidad es: $puntuajeIdentidad")  
}
```

ARTESCO

41

1) Gráfico



2) Identificación Total

1. Application Layer: Contiene las aplicaciones del sistema y las aplicaciones instaladas por el usuario, como teléfono, mensajes, cámara y aplicaciones de terceros.
2. Application Framework: Proporciona servicios y APIs de alto nivel para los desarrolladores. Incluye Activity Manager, Window Manager, Content Providers, Resource Manager y Notification Manager.
3. Android Runtime (ART): Es el entorno de ejecución de Android. Ejecuta las aplicaciones Android y las administra la memoria mediante Garbage Collection. Incluye las core libraries, que proporcionan clases fundamentales de Java y Kotlin utilizadas por las aplicaciones.
4. Native C/C++ Libraries + HAL: contiene bibliotecas nativas como SQLite, OpenGL ES, Media Framework y Webkit. La HAL actúa como intermediaria entre el Hardware y el framework de Android.
5. Linux Kernel: Es la capa base del SO. Gestiona procesos, memoria, seguridad, controladores de dispositivos, red y administración de energía.